ChessLens: Project Document

Real-time digitization and live broadcast of physical chess games using computer vision (no DGT board)

October 2026

1. Executive Summary

ChessLens is a system that watches a normal physical chessboard through an ordinary camera (a phone used as a webcam is enough) and turns the game into moves, a PGN file and a live web broadcast. It needs no sensor board such as DGT.

The project has two goals:

1. **Portfolio goal:** a complete, well-tested project that shows computer vision, backend, frontend, DevOps, CI/CD and cloud skills.
2. **Product goal (optional):** a low-cost alternative for small tournaments, academies and clubs.

Key design decision: the system *tracks moves* instead of recognizing every piece in every frame. Because the starting position and the rules of chess are known, the system only has to work out which *legal* move produced the observed change. This is far more robust than classifying pieces from scratch.

Item	Decision
Name	ChessLens
Dev machine	Windows 11
Camera	Phone as webcam (DroidCam or IP Webcam), also video files
Repo	Public GitHub repo
Cloud	AWS Free Plan, created late (about day 10), small and teardown-friendly
Timeline	1 to 2 weeks (MVP scope)
Real footage	Available (the author has a physical board)

2. Market and Competitive Landscape

The idea already exists, so the project should be positioned honestly rather than as a first-of-its-kind invention.

Player	What it is	Notes
idChess	Phone-based game recognition and live broadcasting for any chessboard	Mature, closed product, reported in use at FIDE events and in 150+ countries. The main competitor.
Chessworld AI	Indian startup (Bengaluru, founder Damodar Mundrikeri)	Started with scoresheet-to-PGN digitization and analytics; piloted camera-based live capture at a state under-7 championship; seeking angel investors and partners. Camera capture looks early-stage.
Chessnut Vision	Hardware-oriented system	Different use case (connecting physical boards to online platforms).
Open source and research	GitHub projects, university papers	Mostly demos. Techniques such as frame differencing and perspective warping are well known.

2.1 Where the gaps are

- **Price and business model:** a cheaper per-event or per-club option for small tournaments and academies.
- **Robustness:** reliable handling of blitz, sloppy piece placement, hands over the board and low camera angles.
- **Region and language support** and local payment methods.
- **Bundles:** live broadcast plus automatic PGN, coach reports and parent summaries.

---

3. Patent and IP Notes

Important: this is *not* a legal freedom-to-operate search. A web search did not turn up any relevant chess-recognition patent. The one loosely related patent found (US 8,328,613, “Game tower”) covers a different product: a camera-assisted interactive board game.

What the search did show is substantial **prior art** in research papers and open source code: comparing a previous frame (with a known game state) to the current frame to find the move. That means the *basic* method (camera, perspective warp, frame differencing, move detection) is public, and a broad patent on it is very unlikely. Risk would come from narrower, newer claims.

How to check properly (free):

1. Google Patents (patents.google.com): search “chess move recognition camera”, “chessboard image move detection”, “game state tracking chess video”.
2. Espacenet (EPO) and WIPO PATENTSCOPE for international filings; also your national patent office search tool.
3. Search by company name: idChess, DGT, Chessnut, Square Off.
4. Check classification codes for chess and electric board games (for example CPC A63F 3/02 and related codes) and image analysis (G06T, G06V). Verify exact codes yourself.
5. Before commercial launch, pay an IP attorney for a short search.
6. Check trademark availability for the name “ChessLens” and the domain name.
7. Check licenses of any open source code or datasets you reuse; some are non-commercial.

---

4. Business Options

Option	Assessment
Sell the tech to Chessworld AI	Unlikely as an acquisition: they are small, building their own camera tech, and looking for investment. Possible as a **partnership, paid pilot, licensing or job**, especially with a working demo and real accuracy numbers.
Start your own product	Stronger option, provided you choose a narrow target segment instead of competing with idChess feature for feature.
Portfolio only	Lowest risk, still very valuable, and does not rule out the other options later.

Recommended path

1. Build the MVP and measure real accuracy.
2. Interview 10 organizers, coaches or academy owners about what they use, what it costs and what annoys them.
3. Run a free pilot at one or two local events.
4. Then decide: charge per event or per month, approach Chessworld AI or idChess with a demo, or keep it as a portfolio project.

This is general guidance, not legal or financial advice.

---

5. Product Scope

5.1 MVP (1 to 2 weeks)

Must have (days 1 to 7)

1. Repository scaffold, `CLAUDE.md`, `.env.example`, README skeleton.
2. Edge client: calibration, stability detection, move detection by tracking with legal-move validation, uncertainty flagging, operator controls (undo, set FEN, force move), offline PGN output, input from phone webcam or video file.
3. Synthetic data generator and an evaluation script that reports accuracy.
4. Server: FastAPI, SQLite, game endpoints, ingest and viewer WebSockets.
5. Web app: live board, move list, PGN download, auto-reconnect, mobile friendly.
6. `docker compose up` runs everything locally.
7. GitHub Actions CI: lint, tests, accuracy regression, Docker build.

Should have (days 8 to 11)

8. Real-footage evaluation with honest numbers, kept separate from synthetic numbers.
9. Simple CD: one small free-plan-eligible EC2 instance via Terraform, frontend on a free static host, billing alarm and a teardown runbook.

Could have (only if time remains)

10. Stockfish evaluation bar, security scans (Trivy, tfsec), CloudFront, a small CNN for automatic recovery from desync.

5.2 Explicitly out of scope

User accounts, multi-board tournaments, payments, mobile native apps, and cloud-side video processing. Anything cut is recorded in `docs/ROADMAP.md`.

---

6. System Architecture

Vision runs on the **edge machine** (the venue laptop), not in the cloud. Only small move events are sent to the server. This keeps cloud costs near zero and latency low.



6.1 Repository layout

```
chesslens/
  edge/        Python client: camera -> moves
  server/      FastAPI backend
  web/         React + TypeScript + Vite spectator app
  infra/       Terraform (AWS)
  tests/       fixtures, synthetic generator, evaluation scripts
  docs/        architecture, runbook, costs, accuracy, roadmap, resume
  .github/workflows/   ci.yml, cd.yml
  CLAUDE.md    conventions and commands for Claude Code
  docker-compose.yml
```

---

7. Component Specifications

7.1 Edge client (`edge/`)

Feature	Details
Calibration	User clicks the 4 board corners once; saved to JSON; choose which side White is on. Warp to an 800 x 800 top-down image using `cv2.getPerspectiveTransform`. Manual corners are used because automatic corner detection fails once pieces are on the board.
Stability detection	Process a frame only after the board has been still for about 0.5 to 1 second (configurable). Ignore frames with hands or other occlusion.
Move detection	Keep a `python-chess` board. For each legal move, compute the expected set of changed squares (2 normally, 4 for castling, 3 for en passant). Score each against the measured per-square change between the last stable frame and the new stable frame. Choose the best, requiring a minimum confidence and a margin over the runner-up.
Ambiguity handling	If ambiguous, **do not guess**: mark the move “uncertain” and wait for operator confirmation.
Promotion	Default to queen; operator can override.
Operator controls	Undo, set position from FEN, force a move, pause and resume, flip board, new game (keyboard plus a minimal local UI).
Networking	Authenticated WebSocket with automatic reconnect and a local queue so no moves are lost on network drops.
Offline mode	Works fully locally, writing PGN, without the server.
Inputs	Camera index, RTSP/HTTP URL (phone apps), or a video file.
Stretch	Small per-square classifier (13 classes: empty plus 6 piece types in 2 colors) to recover automatically from desync.

7.2 Server (`server/`)

- FastAPI with Pydantic models; SQLAlchemy with SQLite by default (Postgres-compatible).
- REST endpoints: create and list games, game detail, PGN download, health check.
- WebSockets: `/ws/ingest/{game_id}` (token auth, edge only) and `/ws/games/{game_id}` (public viewers). Single-instance in-memory publish/subscribe is enough; Redis is not needed.
- Optional Stockfish evaluation with strict depth and time limits, behind a feature flag.
- CORS configuration, input validation, basic rate limiting, structured logging.
- Tests with pytest (unit and WebSocket integration).

7.3 Web app (`web/`)

- Live game page: board (chessground or react-chessboard), move list, status, result, PGN download, optional evaluation bar, automatic reconnect.

- Game list page.
- Responsive, with good phone support.
- Built as static files so it can be hosted on Cloudflare Pages, Vercel, GitHub Pages or S3.
- Vitest tests, ESLint, strict TypeScript.

---

8. Vision Pipeline in Detail

1. **Camera setup:** fixed camera, overhead or steep angle, even light, no glare.
2. **Calibration:** four clicked corners produce a perspective transform; every frame becomes a flat top-down board image.
3. **Square extraction:** split the warped image into 64 crops (100 x 100 pixels each).
4. **Stability gate:** compare consecutive frames; proceed only when the board has settled.
5. **Change scoring:** compare the last stable board with the new one and compute a change score per square (for example, mean absolute difference or structural similarity, with lighting normalization).
6. **Move matching:** for every legal move, build the expected changed-square set and compare it with the measured scores; pick the best match with a confidence threshold and a margin.
7. **State update:** push the move onto the `python-chess` board, store the new stable frame as the reference.
8. **Recovery:** operator can undo or set the position from FEN; optional CNN-based re-read of the full board as a stretch goal.

8.1 Known failure modes

Problem	Mitigation
Desync after a misread move	Undo / set-FEN controls; ambiguity flagging; later, CNN recovery
Promotion piece unknown	Default queen; operator override
Glare on glossy boards	Matte board, diffuse lighting, camera placement
Hands covering the board	Stability gate
Pieces knocked over or placed off-center	Uncertain flag; operator correction
Low camera angles	Overhead or steep mounting; tests with several angles
Similar colors between pieces and squares	Lighting normalization; contrast checks during calibration

---

9. Testing and Evaluation

Testing is the most important part of the project, because honest measured accuracy is what makes it credible.

9.1 Synthetic data

- Render boards from `python-chess` positions (Pillow) from known PGNs.
- Apply perspective warp, lighting changes, noise and simulated hand occlusion.
- Produce video fixtures with known ground-truth moves.

9.2 Evaluation script (`tests/eval.py`)

- Runs the edge pipeline on fixtures.
- Reports move accuracy per game and overall.
- Breaks failures down by category: castling, en passant, promotion, ambiguity, occlusion.
- CI fails if synthetic accuracy falls below a configured threshold.

9.3 Real footage

- Record 10 to 20 clips or short games, including castling, captures, a promotion, fast moves and sloppy placement.

- Label ground truth (PGN) for each clip. Large video files are git-ignored (`tests/real/README.md` explains how to record and label).
- Run the same evaluation and report results in `docs/ACCURACY.md`, **synthetic and real numbers kept separate**.

9.4 Rules for honesty

- Never publish a metric that the evaluation script did not produce.
- Resume and README numbers must be traceable to a script run.

---

10. DevOps and CI/CD

10.1 Containers

- Multi-stage, non-root Dockerfiles for server and web.
- `docker-compose.yml` for local development and for single-VM deployment (server plus nginx).

10.2 GitHub Actions

`ci.yml` (every pull request)

1. Lint: ruff (Python) and ESLint (web).
2. Type checks: mypy (best effort) and TypeScript strict mode.
3. Tests: pytest and Vitest.
4. Accuracy regression on synthetic fixtures (fails below threshold).
5. Docker build.
6. Security scan with Trivy; `terraform fmt`, `terraform validate`, tfsec or checkov.

`cd.yml` (merge to main)

1. Build and push images to GitHub Container Registry (GHCR).
2. Deploy backend to EC2 via SSM or SSH.
3. Deploy frontend to the chosen static host.
4. Run smoke tests.
5. Manual approval gate for production.

Authenticate to AWS with **GitHub OIDC**, not long-lived access keys. GitHub Actions is free for public repositories.

10.3 Infrastructure as Code (`infra/`)

Terraform for: a small EC2 instance (confirm free-plan eligibility), security group, IAM role for SSM, an S3 bucket for the frontend (CloudFront only if allowed on the free plan), an AWS Budget alert, and the GitHub OIDC role. Include a `terraform destroy` runbook.

10.4 Monitoring

Health endpoint, container logs, CloudWatch basic metrics, and one billing alarm.

---

11. AWS Free Plan and Cost Plan

11.1 Facts about the current AWS Free Tier

- New accounts get **$100 in credits at signup** and can earn **another $100** by completing five service activities (about $20 each), for up to $200 total.
- At signup you choose a **Free plan** or a **Paid plan**.
  - **Free plan:** no charges unless you upgrade; it **expires after 6 months or when credits run out, whichever is first**; the account then closes and resources and data are lost (there is a short window to upgrade). Not all

AWS services are available on it.
    - **Paid plan:** credits are applied automatically, access to all services, and you are billed beyond the credits.
- A credit card is required even for the Free plan.
- Accounts created before July 15, 2025 stay on the older Free Tier program.
- Terms change, so always check the official AWS Free Tier page before relying on them.

11.2 Timing

The 6-month clock starts at signup, so **create the AWS account only when you reach the deployment phase** (about day 10).

11.3 Budget-friendly architecture

Component	Choice
Vision processing	The author's own laptop (not the cloud)
Backend	One small EC2 instance running Docker Compose (FastAPI, nginx, SQLite or Postgres in a container)
Frontend	S3 static hosting, or Cloudflare Pages, Vercel or GitHub Pages
Image registry	GitHub Container Registry (avoids ECR charges)
CI/CD	GitHub Actions (free for public repos)
IaC	Terraform
Monitoring	CloudWatch basic metrics and a billing alarm, or Uptime Kuma in a container
Chess engine	Stockfish (open source)

Avoid: NAT gateways, load balancers, RDS, and Fargate. They are the usual causes of surprise charges.

11.4 Cost-protection habits

1. Set an AWS Budget alert on day one of having the account.
2. Enable MFA on the root user.
3. Stop or `terraform destroy` resources when not demoing.
4. Keep everything in Git so the stack can be recreated in minutes.
5. Capture screenshots and a demo video of the live deployment before the account closes.
6. If an alternative free host is preferred (for example, Oracle Cloud Always Free, Render or Fly.io), check its current limits first.

---

12. Timeline (1 to 2 Weeks)

Days	Work
1	Install tools, create the GitHub repo, test the phone camera in OpenCV, paste the master prompt, answer Phase 0 questions, approve `PLAN.md`
2 to 4	Vision core, synthetic data generator, evaluation script (hardest part)
5 to 6	Server, web app, docker compose, CI
7	**Record real games** (castling, captures, promotion, fast moves)
8 to 9	Real-footage evaluation, fixes, honest README
10 to 11	Create AWS account, Terraform deploy, smoke test, record demo, tear down

12+	Demo video, resume, buffer

If time runs short, drop the AWS deployment. A project that works locally, with tests, CI and honest accuracy numbers, is better than a half-working cloud deployment. Put the cloud work on the roadmap.

---

13. Windows Setup

1. Install **Python 3.11+**, **Node 20 LTS**, **Git**, **Docker Desktop** (with WSL2) and **VS Code**.
2. Install **Terraform** and the **AWS CLI** (needed later).
3. Install **Claude Code** following Anthropic's current official documentation (verify the Windows steps there).
4. Install **DroidCam** or **IP Webcam** on the phone and confirm the video opens in OpenCV (`cv2.VideoCapture`) before building anything else.
5. Keep the project at a short path such as `C:\dev\chesslens`, **not inside OneDrive** (syncing can corrupt Git repos and slow builds).
6. Use PowerShell-friendly commands; scripts should work on Windows or come with both PowerShell and bash versions.

---

14. Your Checklist

Before starting

- [ ] Install the tools in Section 13.
- [ ] Create a GitHub account and an empty public repository.
- [ ] Test the phone as a webcam.
- [ ] Prepare the board: plain surface, overhead or steep mount (tripod or stack of books), even light, no glare.

During the build

- [ ] Fill in the decisions block and answer the Phase 0 questions.
- [ ] Review `PLAN.md` before approving it.
- [ ] Record 10 to 20 real clips with ground-truth PGNs.
- [ ] Review each pull request; you must be able to explain every part in interviews.
- [ ] Create the AWS account only when asked (around day 10), enable MFA, set a budget alert.
- [ ] Add GitHub secrets and the AWS OIDC role when Claude Code gives the steps.
- [ ] Approve each `terraform apply` yourself.

After it works

- [ ] Run the real-footage evaluation and keep the honest numbers.
- [ ] Record a 2-minute demo video; add screenshots to the README.
- [ ] Do the patent and trademark checks if you plan to sell.
- [ ] Update the resume with measured results only.
- [ ] Run `terraform destroy` when finished demoing.

---

15. Risks and Mitigations

Risk	Mitigation
Real-world accuracy is lower than synthetic accuracy	Evaluate on real clips early (day 7), report both, add operator controls
Scope too large for 1 to 2 weeks	Follow MVP priorities; drop AWS first, then stretch goals
AWS cost surprises or account closure	Free plan, budget alert, teardown runbook, everything in Git

Strong competitor (idChess)	Position on price, region, blitz robustness and bundles; validate with real users
IP or licensing issue	Patent and trademark search; check licenses of reused code and datasets
Over-reliance on generated code	Review every pull request; keep code small and tested
Desync after one misread move	Ambiguity flagging, undo, set-FEN, optional CNN recovery

---

16. Resume Entry

ChessLens: Real-Time Chess Digitization and Live Broadcast *(Oct. 2026 to Present) Python, OpenCV, FastAPI, React.js, Docker, GitHub Actions, AWS*

- Developed a computer-vision system that converts live camera video of a physical chessboard into moves and PGN without a sensor board, using perspective calibration, frame differencing and legal-move validation.
- Streamed live games to a React spectator page via a FastAPI WebSocket backend; containerized with Docker and automated testing and deployment with a GitHub Actions CI/CD pipeline to AWS (EC2, S3).
- **Additional technologies:** python-chess, WebSockets, SQLite, Nginx, GHCR, Terraform.

Rules: add a measured number only after the evaluation script produces it (for example, “achieving X% move accuracy over N real games”). Remove any technology you did not actually use (for example, drop AWS and Terraform if you skip deployment).

---

17. Master Prompt for Claude Code

Paste everything inside the block below into Claude Code, opened in your project folder.

```
# ROLE
You are a senior engineer building "ChessLens" end to end with me. ChessLens
turns live camera video of a normal physical chessboard (no DGT/sensor board)
into moves, PGN and a live web broadcast. It is a portfolio project with
real-product ambitions, so quality, tests and honesty matter more than speed.

# MY DECISIONS
- OS: Windows 11 (use PowerShell commands and Windows-safe paths; Docker
  Desktop with WSL2; avoid bash-only scripts, or provide both)
- Camera: phone as webcam via DroidCam or IP Webcam (support both a cv2
  VideoCapture index and an RTSP/HTTP URL; also support reading a video file)
- AWS: I have NO account yet. Do not touch AWS until I tell you I have
  created one. Until then, everything runs locally.
- Frontend hosting: undecided. Build the web app as static files that can go
  to Cloudflare Pages, Vercel or S3 later.
- Repo visibility: public (no secrets, ever)
- Real board available for footage: yes
- Project name: ChessLens
- Time available: 1-2 weeks, so follow the MVP SCOPE below.

# OPERATING RULES
1. Start with Phase 0. Ask all blocking questions in ONE batch, then write
   PLAN.md and wait for my approval before writing code.
2. Work phase by phase. At the end of each phase: run all tests, commit with
   conventional commits, summarize what works and what doesn't, then continue
   unless a step needs me.
3. STOP and give me exact numbered instructions whenever a step needs my
   action: AWS credentials, GitHub secrets/settings, camera setup, recording
   footage, or any `terraform apply`. Never run `terraform apply` or create
   cloud resources without my explicit confirmation.
4. Never fabricate metrics. Accuracy numbers appear in the README only if they
   are produced by the evaluation script, labelled synthetic vs real.
5. Never commit secrets. Provide `.env.example`. Use GitHub OIDC for AWS, not
   long-lived keys.
6. Free-tier only. Do not use NAT gateways, load balancers, RDS or Fargate.
   Before any infra change, show an estimated monthly cost. Verify current
   AWS Free Plan restrictions and eligible instance types from official AWS
   docs rather than assuming.
7. Prefer simple, readable code, type hints, and small functions. Create a
```

```
   CLAUDE.md in the repo recording conventions and commands.
8. If a requirement is ambiguous, state your assumption in PLAN.md rather than
   guessing silently.

# ARCHITECTURE
Monorepo:
  edge/    Python client that runs on the venue laptop: camera -> moves
  server/  FastAPI backend: ingest, store, broadcast
  web/     React + TypeScript + Vite spectator app
  infra/   Terraform (AWS)
  tests/   fixtures and evaluation scripts
  docs/    architecture, runbook, costs, accuracy, roadmap, resume
  .github/workflows/  CI and CD

Vision runs on the edge machine, NOT in the cloud. Only move events are sent
to the server.

## edge/ requirements
- Calibration: user clicks 4 board corners; save to JSON; choose which side
  white is on. Warp each frame to an 800x800 top-down board with
  cv2.getPerspectiveTransform. Do not rely on findChessboardCorners.
- Stability: process a frame only after the board has been still for ~0.5-1s
  (configurable); ignore frames with hands/occlusion.
- Move detection by TRACKING, not full piece recognition: keep a python-chess
  Board. For each legal move compute the expected set of changed squares
  (2 normally, 4 for castling, 3 for en passant), score each against the
  measured per-square change between the last stable and new stable frame,
  and pick the best with a minimum confidence and margin over the runner-up.
- If ambiguous, do NOT guess: mark the move "uncertain" and wait for operator
  confirmation.
- Promotion defaults to queen; operator can override.
- Operator controls (keyboard + minimal local UI): undo, set position from
  FEN, force a move, pause/resume, flip board, new game.
- Send events to the server over an authenticated WebSocket with automatic
  reconnect and a local queue so no moves are lost on network drops.
- Offline mode: works fully locally without the server.
- Stretch (only after everything else is done): a small per-square classifier
  to auto-recover from desync.

## server/ requirements
- FastAPI + Pydantic. SQLite by default (SQLAlchemy), Postgres-compatible.
- Endpoints: create/list games, game detail, PGN download, health check.
- WebSocket: /ws/ingest/{game_id} (token auth, edge only) and
  /ws/games/{game_id} (public viewers). Single-instance in-memory pub/sub
  is fine; no Redis.
- Optional Stockfish evaluation with strict depth/time limits, behind a flag.
- CORS config, input validation, rate limiting, structured logging.
- Tests with pytest (unit + WebSocket integration).

## web/ requirements
- Live game page: board (chessground or react-chessboard), move list, status,
  result, PGN download, optional eval bar, auto-reconnect.
- Game list page. Responsive and works well on a phone.
- Tests with Vitest; lint with ESLint; strict TypeScript.

# TESTING AND EVALUATION (core of the project)
- Build a synthetic data generator: render boards from python-chess with
  Pillow, then apply perspective warp, lighting jitter, noise, and simulated
  hand occlusion. Produce video fixtures from known PGNs.
- Write tests/eval.py: run the edge pipeline on fixtures and report move
  accuracy per game and overall, plus failure categories (castling, en
  passant, promotion, ambiguity, occlusion).
- CI must fail if synthetic accuracy drops below a configured threshold.
- Create tests/real/README.md explaining how I record and label real clips
  (large files git-ignored). Run the same eval on real clips once I add them
  and report the numbers separately in docs/ACCURACY.md.

# DEVOPS
- Dockerfiles (multi-stage, non-root) for server and web; docker-compose for
  local development and for the single-VM deployment (server + nginx).
- GitHub Actions:
  ci.yml (PRs): ruff, mypy (best effort), pytest, ESLint, Vitest, build,
    accuracy regression, Docker build, Trivy scan, terraform fmt/validate
    and tfsec/checkov.
  cd.yml (main): build and push images to GHCR, deploy backend to EC2
    (SSM or SSH), deploy frontend, run a smoke test, with a manual approval
    gate for production.
- Terraform (infra/): small EC2 instance (verify free-plan eligibility),
  security group, IAM role for SSM, S3 bucket for the frontend (+ CloudFront
  only if free-plan allowed), AWS Budget alert, GitHub OIDC role. Include a
```

```
  `terraform destroy` runbook.
- Monitoring: health endpoint, container logs, CloudWatch basic metrics, and
  one billing alarm.

# MVP SCOPE (overrides PHASES and the earlier requirements where they conflict)
Priority order. Finish each before starting the next; stop when time runs out.

MUST (days 1-7):
1. Repo scaffold, CLAUDE.md, .env.example, README skeleton.
2. Edge: calibration, stability detection, move detection by tracking with
   legal-move validation, uncertainty flagging, undo / set-FEN / force-move
   controls, offline PGN output, works from a phone-webcam or a video file.
3. Synthetic data generator + tests/eval.py with accuracy report.
4. Server: FastAPI, SQLite, game endpoints, ingest + viewer WebSockets.
5. Web: live board, move list, PGN download, auto-reconnect, mobile-friendly.
6. docker-compose up runs everything locally.
7. GitHub Actions CI: lint, tests, accuracy regression, Docker build.

SHOULD (days 8-11):
8. Real-footage evaluation (I record clips) + docs/ACCURACY.md with
   synthetic and real numbers kept separate.
9. Simple CD: deploy to one small free-plan-eligible EC2 via Terraform + SSM,
   frontend to a free static host. Ask me before any apply. Provide a
   terraform destroy runbook and a billing alarm.

COULD (only if time remains):
10. Stockfish evaluation bar, Trivy/tfsec scans, CloudFront, CNN-based
    desync recovery.

Cut aggressively: no auth beyond a shared ingest token, no user accounts,
no multi-board support. Record anything cut in docs/ROADMAP.md.

# PHASES
0. Questions + PLAN.md (stack, assumptions, risks, cost estimate). Wait for OK.
1. Repo scaffold, tooling, CLAUDE.md, pre-commit, .env.example, LICENSE (ask me).
2. Synthetic data generator + edge vision core + unit tests + eval script.
3. Server + tests.
4. Web app + tests.
5. Edge <-> server <-> web end-to-end, runnable with `docker compose up`.
6. CI workflow, green on a PR.
7. Terraform + CD. STOP and walk me through AWS and GitHub setup.
8. Real-footage evaluation, README (mermaid architecture diagram, setup,
   demo GIF placeholders, limitations), docs/RUNBOOK.md, docs/COSTS.md,
   docs/ACCURACY.md, and a short demo script.

# DEFINITION OF DONE
- `docker compose up` runs the full stack locally.
- Edge pipeline plays a recorded or synthetic game and spectators see it live.
- CI green; CD deploys to AWS with approval; teardown is documented.
- README states honest accuracy numbers (synthetic and real, separately)
  and known limitations.
- Resume bullets in docs/RESUME.md using only measured numbers.

Begin with Phase 0 now.
```

---

18. Sources and References

Facts about competitors and AWS came from public web pages found in October 2026. Re-check them before relying on them, since products and pricing change.

- idChess: ChessBase India article “New technology in Chess: idChess” and the FIDE article on idChess at the 2025 World Schools Team Chess Championship (fide.com).
- Chessworld AI: ChessBase India article “How Chessworld AI is building the future of chess analytics” (chessbase.in).
- Chessnut Vision: chessnutech.com blog post on the Vision hardware system.
- Research and open source: “A Computer Vision System for Chess Game Tracking” (CVWW 2016); “LiveChess2FEN” (arXiv 2012.06858); “End-to-End Chess Recognition” (arXiv 2310.04086); GitHub projects such as chess-agi and chess-video-move-detection.
- Patent found (loosely related): US 8,328,613, “Game tower” (USPTO).
- AWS Free Tier: aws.amazon.com/free and the AWS announcement “AWS Free Tier now offers $200 in credits and 6-month free plan” (July 2025).

This document is for planning purposes. It is not legal, financial or patent advice.